

Practical No: - CASE STUDIES

ON

Cryptography and Network Security - SSCS 4020

TITLE: Cryptography & Network Security – Practical Assignment

BACHLEOR OF SCIENCE IN INFORMATION TECHNOLOGY
(HONOURS.)

SUBMITTED BY:

Name: Shah Rishabh Alpeshbhai

Enrolment: 23SS02IT181

Class: BSCIT-H – 7A - Batch 2023-27

Practical 1: Implementation of Diffie-Hellman Key Exchange using Python

Theory :

Diffie-Hellman Key Exchange is a cryptographic method used to securely generate a **shared secret key** between two users over an insecure network.

In this method, the actual secret key is **never directly sent** from one user to another.

- First, two public values, **prime number (p)** and **generator (g)**, are selected.
- Both users choose their own **private keys**.
- Each user calculates a **public key** using their private key.
- The public keys are exchanged between the two users.
- Each user then uses the received public key and their own private key to calculate the **shared secret key**.
- Both users get the **same shared secret key**.
- This shared key can be used for secure communication or encryption.
- The security of Diffie-Hellman is based on the difficulty of solving the **discrete logarithm problem**.

Thus, Diffie-Hellman provides a way to establish a common secret key without transmitting the secret key directly.

Code :

```
# Taking public values from user
p = int(input("Enter a prime number (p): "))
g = int(input("Enter a primitive root / generator (g): "))

# Taking private keys from users
alice_private = int(input("Enter Alice's private key: "))
bob_private = int(input("Enter Bob's private key: "))

# Calculate public keys
alice_public = pow(g, alice_private, p)
bob_public = pow(g, bob_private, p)

print("\n--- Public Keys ---")
print("Alice's Public Key:", alice_public)
print("Bob's Public Key:", bob_public)
```

```
# Calculate shared secret keys

alice_secret = pow(bob_public, alice_private, p)
bob_secret = pow(alice_public, bob_private, p)

print("\n--- Shared Secret Keys ---")
print("Alice's Shared Secret Key:", alice_secret)
print("Bob's Shared Secret Key:", bob_secret)

# Verify shared keys
if alice_secret == bob_secret:
    print("\nDiffie-Hellman Key Exchange Successful!")
else:
    print("\nKey Exchange Failed!")
```

Output :

```
Enter a prime number (p): 15
Enter a primitive root / generator (g): 4
Enter Alice's private key: 5
Enter Bob's private key: 20

--- Public Keys ---
Alice's Public Key: 4
Bob's Public Key: 1

--- Shared Secret Keys ---
Alice's Shared Secret Key: 1
Bob's Shared Secret Key: 1

Diffie-Hellman Key Exchange Successful!
```

Practical 2: Simulation of a Digital Signature using Python

Theory :

A **Digital Signature** is a cryptographic technique used to verify the **authenticity and integrity** of a message or document.

It helps the receiver check whether the message was sent by the correct sender and whether the message was changed during transmission.

In this practical, we use **RSA and SHA-256** to simulate a digital signature.

- The user enters a message.
- A SHA-256 hash is generated from the message.
- The hash is signed using the sender's **private key**.
- The receiver uses the sender's **public key** to verify the signature.
- If the verification is successful, the message is considered authentic and unchanged.
- If the message is modified, signature verification fails.

Code :

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
# Generate RSA private key
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
# Generate public key
public_key = private_key.public_key()
# Take message from user
message = input("Enter message: ")
# Convert message into bytes
message_bytes = message.encode()
# Create digital signature
signature = private_key.sign(
    message_bytes,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
```

```
),  
hashes.SHA256()  
)  
print("\nOriginal Message:", message)  
print("Digital Signature Generated Successfully!")  
print("Signature Length:", len(signature), "bytes")  
# Verify digital signature  
try:  
    public_key.verify(  
        signature,  
        message_bytes,  
        padding.PSS(  
            mgf=padding.MGF1(hashes.SHA256()),  
            salt_length=padding.PSS.MAX_LENGTH  
        ),  
        hashes.SHA256()  
    )  
    print("Digital Signature Verified Successfully!")  
except Exception:  
    print("Digital Signature Verification Failed!")
```

Output :

```
Enter message: Hello Rishabh Shah  
  
Original Message: Hello Rishabh Shah  
Digital Signature Generated Successfully!  
Signature Length: 256 bytes  
Digital Signature Verified Successfully!
```

Practical 3: Demonstrate Email Encryption using the PGP Tool

Theory

PGP (Pretty Good Privacy) is a security technique used to protect emails and digital messages. It uses encryption to keep the message secure during communication.

PGP uses a **public key and private key**:

- The **public key** is used to encrypt the message.
- The **private key** is used to decrypt the message.
- The public key can be shared with other users.
- The private key must be kept secret.
- PGP helps provide **confidentiality, authentication, and integrity**.
- PGP can be used to encrypt messages and also create digital signatures.

For this practical, **Gpg4win with Kleopatra** can be used as the PGP tool.

Procedure / Steps

Step 1: Install Gpg4win

Download and install **Gpg4win** on the computer.

Open **Kleopatra** after installation.

[Gpg4win Official Website](#)

Step 2: Create a Key Pair

1. Open **Kleopatra**.
2. Click **File → New Key Pair**.
3. Select **Create a personal OpenPGP key pair**.
4. Enter your **Name**.
5. Enter your **Email Address**.
6. Set a secure password/passphrase.
7. Click **Create Key**.

Two keys are generated:

Public Key

Private Key

Step 3: Export the Public Key

1. Select your generated certificate.
2. Right-click on it.

3. Select **Export**.
4. Save the public key file.

The public key can be shared with the receiver.

Step 4: Import the Receiver's Public Key

1. Open Kleopatra.
2. Click **Import**.
3. Select the receiver's public key file.
4. Click **Open/Import**.

The receiver's public key is now available for encryption.

Step 5: Create the Message

Create a text file named:

message.txt

Enter a message such as:

Hello,

This is a confidential message.

This message is encrypted using PGP.

Regards,

Rishabh

Step 6: Encrypt the Message

1. Right-click on message.txt.
2. Select the **Gpg4win/Kleopatra encryption** option.
3. Select **Encrypt**.
4. Select the receiver's public key.
5. Click **Encrypt**.

The message will now be converted into an encrypted PGP message/file.

Step 7: Send the Encrypted Message

The encrypted file can be sent to the receiver through email or another communication channel.

The **private key should never be shared**.

Step 8: Decrypt the Message

On the receiver's computer:

1. Open the encrypted file.
2. Select **Decrypt/Verify**.
3. Enter the private-key password when requested.
4. The message will be decrypted.
5. Open the decrypted file.

The original message will be recovered.

Code:

```
import pgpy

from pgpy.constants import PubKeyAlgorithm, KeyFlags, HashAlgorithm
from pgpy.constants import SymmetricKeyAlgorithm, CompressionAlgorithm

# Create PGP key pair
key = pgpy.PGPKey.new(
    PubKeyAlgorithm.RSAEncryptOrSign,
    2048
)

uid = pgpy.PGPUID.new(
    "Rishabh",
    email="rishabh@example.com"
)

# Add user information to the key
key.add_uid(
    uid,
    usage={
        KeyFlags.Sign,
        KeyFlags.EncryptCommunications,
        KeyFlags.EncryptStorage
    },
    hashes=[HashAlgorithm.SHA256],
```



```
ciphers=[SymmetricKeyAlgorithm.AES256],
compression=[CompressionAlgorithm.ZLIB]
)
# Create public key
public_key = key.pubkey
# Take message from user
message_text = input("Enter your confidential message:\n")
# Display original message
print("\n1. Original Message")
print("=====")
print(message_text)
# Create PGP message
pgp_message = pgpy.PGPMessage.new(message_text)
# Encrypt message using public key
encrypted_message = public_key.encrypt(pgp_message)
# Display encrypted message
print("\n2. Encrypted Message")
print("=====")
print(encrypted_message)
# Decrypt message using private key
decrypted_message = key.decrypt(encrypted_message)
# Display decrypted message
print("\n3. Decrypted Message")
print("=====")
print(decrypted_message.message)
# Verify encryption and decryption
if decrypted_message.message == message_text:
    print("\nPGP Encryption and Decryption Successful!")
else:
    print("\nDecryption Failed!")
```

Output

1. Original Message

Hello,

This is a confidential message.

This message is encrypted using PGP.

Regards,

Rishabh

2. Encrypted Message

The output will appear as an encrypted PGP message/file, for example:

-----BEGIN PGP MESSAGE-----

[Encrypted PGP Data]

-----END PGP MESSAGE-----

3. Decrypted Message

After successful decryption:

Hello,

This is a confidential message.

This message is encrypted using PGP.

Regards,

Rishabh

Practical 4: Case Study on Recent Cyberattacks and Prevention Techniques

Theory

A **cyberattack** is a malicious attempt to access, damage, modify, steal, or disrupt computer systems, networks, or data.

Cyberattacks can include:

- Ransomware attacks
- Phishing attacks
- Malware attacks
- Data theft
- Password attacks
- Social engineering
- Denial-of-Service (DoS) attacks

Organizations need proper security controls, monitoring, backups, authentication, and employee awareness to reduce the risk of cyberattacks.

❖ Case Study: Marks & Spencer Cyberattack – 2025

Organization

Marks & Spencer (M&S) is a major UK retailer.

Attack Period

The cyber incident occurred in **April 2025**.

Type of Attack

The incident involved **unauthorized access to parts of M&S's technology environment** and caused significant operational disruption. M&S took some systems offline as part of its response.

The UK National Cyber Security Centre (NCSC) reported that ransomware attacks against major UK retailers, including M&S, caused significant disruption in 2025.

❖ How the Attack Affected M&S

The cyber incident affected several business operations.

1. Online Shopping Disruption

M&S temporarily paused online orders through its website and app during the incident.

2. Payment Problems

Some payment services, including contactless payments, were temporarily unavailable.

3. Click & Collect

Click & Collect services were temporarily affected.

4. Manual Operations

M&S moved some processes offline and introduced manual processes to maintain business operations.

5. Customer Data

M&S later stated that some personal customer data had been taken. The company said the affected information could include contact details, date of birth, and online order history, while usable payment details and account passwords were not included.

6. Financial Impact

M&S later reported that the incident resulted in **£131.3 million of incident-related costs** in its 2025/26 results.

❖ Prevention Techniques

Organizations can use the following techniques to reduce the risk and impact of cyberattacks.

1. Use Multi-Factor Authentication (MFA)

MFA provides an additional security layer beyond passwords. It should be enabled especially for important accounts and administrative access. CISA and NCSC recommend comprehensive MFA as a key protection.

2. Strong Password Policies

Organizations should use strong and unique passwords and avoid sharing passwords between different accounts.

3. Regular Software Updates

Operating systems, applications, servers, and security software should be regularly updated and patched. CISA specifically recommends prioritizing known exploited vulnerabilities.

4. Regular Backups

Important data should be backed up regularly. Offline or protected backups can help organizations recover after ransomware attacks.

5. Employee Security Awareness

Employees should be trained to identify:

- Phishing emails
- Fake login pages
- Suspicious links
- Social engineering
- Unusual password-reset requests

6. Network Monitoring

Organizations should continuously monitor networks, user accounts, and systems for unusual activity.

7. Access Control

Users should only receive the permissions required for their work. Administrative accounts should receive additional protection.

8. Incident Response Plan

Organizations should have a proper incident response plan so that they can quickly:

- Detect the attack
- Isolate affected systems

- Protect important data
- Restore services
- Investigate the incident
- Improve security after the incident

❖ Lessons Learned

The M&S incident shows that a cyberattack can affect more than just computer systems. It can also affect:

- Online services
- Payments
- Supply chains
- Customer services
- Business operations
- Company finances
- Customer data

The NCSC emphasizes that organizations need not only prevention and detection but also the ability to **continue operating and recover after a cyber incident**.

❖ Result

The case study of the **2025 Marks & Spencer cyber incident** was studied successfully. The attack demonstrated how unauthorized access and cyber threats can disrupt business operations and affect customer data. Prevention techniques such as **MFA, strong passwords, regular patching, backups, access control, security monitoring, employee awareness, and incident response planning** can help reduce the risk and impact of future cyberattacks.

Practical 5: Perform Basic Steganography using Open-Source Tools

Theory

Steganography is a technique of hiding secret information inside another file such as an image, audio, video, or document.

In image steganography, a secret message or file is hidden inside an image called the **cover image**. The resulting image is called the **stego image**.

The main purpose of steganography is to hide the existence of the secret information.

Main Points

- **Cover Image:** Original image used to hide the information.
- **Secret Data:** Message or file that needs to be hidden.
- **Stego Image:** Image containing the hidden information.
- **Extraction:** Process of recovering the hidden information.
- A password can be used to provide additional protection.

Tool Used

OpenStego

OpenStego is an open-source tool that can be used for data hiding and digital watermarking.

Requirements

- Computer/Laptop
- OpenStego
- One image file such as .png or .jpg
- One text file containing the secret message

Example files:

Cover Image: vehicle.png

Secret File: secret.txt

Output Image: stego-image.png

Procedure

Step 1: Install OpenStego

Download and install OpenStego from its official website.

Open the **OpenStego** application.

Step 2: Prepare the Cover Image

Select an image that will be used to hide the secret information.

Example:

vehicle.png

This is the original **cover image**.

Step 3: Create Secret Message

Create a text file named:

secret.txt

Write a message inside it:

This is a confidential message.

Steganography hides this message inside an image.

Regards,

Rishabh

Save the file.

Step 4: Select Embed Data

In OpenStego, select the option for **data hiding / Embed Data**.

Select the cover image.

Step 5: Select Secret File

Select:

secret.txt

as the file to be hidden.

Step 6: Select Output File

Choose a location and name for the output image.

For example:

stego-image.png

Step 7: Enter Password

If required, enter a password for protecting the hidden data.

Example:

Rishabh@123

Keep the password safe because it may be required during extraction.

Step 8: Hide the Data

Click the **Embed Data** or equivalent button.

OpenStego will create the stego image.

Cover Image + Secret File

↓

OpenStego

↓

Stego Image

Step 9: Check Stego Image

Open the generated:

stego-image.png

The image should look similar to the original image.

The secret message is hidden inside the image.

Step 10: Extract the Secret Data

Open OpenStego again and select the **Extract Data** option.

Select:

stego-image.png

Enter the password if one was used.

Select the output location.

Step 11: Verify the Message

Open the extracted secret.txt file.

The original message should be recovered:

This is a confidential message.

Steganography hides this message inside an image.

Regards,

Rishabh

Output

1. Original Cover Image

vehicle.png

This is the original image before hiding the message.

2. Secret Message

This is a confidential message.

Steganography hides this message inside an image.

Regards,

Rishabh

3. Stego Image

stego-image.png

The secret message is hidden inside this image.

4. Extracted Message

This is a confidential message.

Steganography hides this message inside an image.

Regards,

Rishabh

The extracted message is the same as the original